Deep RL for Automated Stock Trading

CS5242 Neural Networks and Deep Learning — Group ID24

Rutwik Shrikant Dharanappagoudar Zhu Yufan Fei Xin

Semester 2, 2025/26

1 Introduction

Stock trading is a sequential decision-making problem where an agent observes market state and chooses to *buy*, *hold*, or *sell* to maximise long-term profit. Classical rule-based strategies are brittle and unable to adapt to non-stationary market dynamics.

Deep Reinforcement Learning (DRL) learns trading policies directly from data without hand-crafted rules. We implement DQN and Double DQN (DDQN) **entirely from scratch** in PyTorch and evaluate them on historical DJIA data against a buy-and-hold baseline. The goal is to deeply understand DRL by designing and analysing these algorithms from first principles.

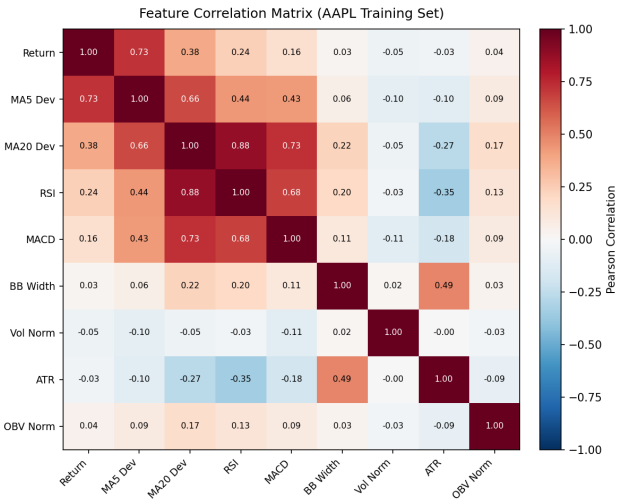

Figure 2: Feature correlation matrix on the AAPL training set. Low cross-correlation indicates complementary signals.

2 Data

We use the `alincijov/trading` Kaggle dataset, which provides daily OHLCV data for 30 DJIA stocks from 2009–2020 (∼2,900 days per ticker), downloaded via `kagglehub`. Data is split chronologically into 70%/15%/15% train/val/test sets.

For each ticker we compute 9 features: (1) daily return; (2–3) short/long MA deviations; (4) RSI-14; (5) MACD signal; (6) Bollinger Band width; (7) normalised volume; (8) ATR; and (9) OBV. All features are z-score normalised using **training-set statistics only** to prevent data leakage. Raw returns and prices are preserved separately for reward computation.

Figure 2 shows the Pearson correlation between all 9 features. Most pairs exhibit low correlation, confirming that each indicator captures distinct market signals. The strongest correlations are between BB Width and ATR (0.63), which is expected as both measure volatility, and between the two MA deviations (0.45).

The full technical indicator time series and feature distribution comparisons across splits are provided in Appendix A (Figures 4 and 5).

3 Environment Design

We implement a Gym-style `TradingEnv` for single-asset trading. The state $s_t \in \mathbb{R}^{10}$ consists of 9 normalised features plus the current position $\text{pos}_t \in \{-1, 0, +1\}$. The agent chooses from three discrete *delta* actions: Decrease (-1), Hold (0), Increase $(+1)$, which increment or decrement the position by one unit, clamped to $[\text{pos}_{\min}, \text{pos}_{\max}]$. This formulation makes partial transitions natural and prevents abrupt position reversals.

The reward is a log-return with transaction cost and

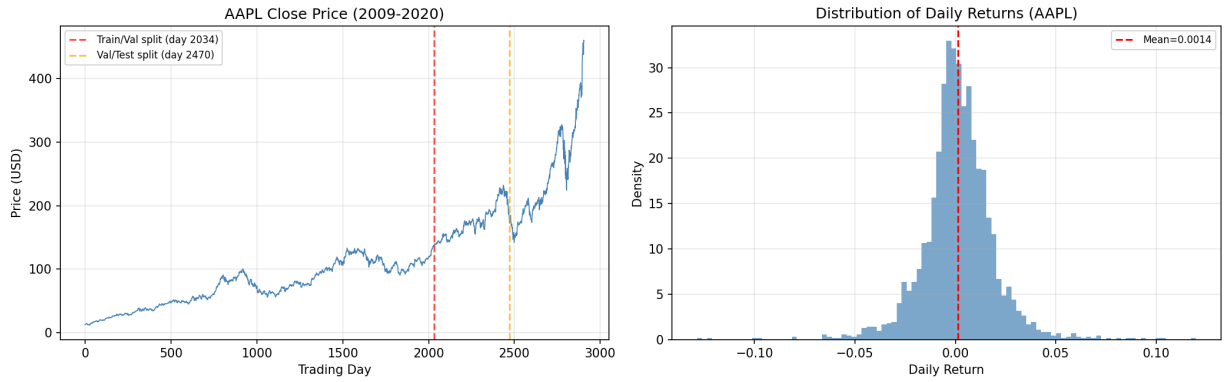

Figure 1: Left: AAPL close price (2009–2020) with train/val/test boundaries. Right: Daily return distribution showing fat tails.

drawdown penalty:

$$r_t = \log \frac{V_t}{V_{t-1}} - c_t \cdot \mathbf{1}[\text{pos}_t \neq \text{pos}_{t-1}] + \lambda \min\Big(0,\ \frac{V_t - V_t^{\text{peak}}}{V_t^{\text{peak}}}\Big) \quad (1)$$

where $\lambda = 0.5$ and c_t is either a flat fee (10 bps) or proportional to position change magnitude. Rewards are clipped to $[-1, 1]$ for training stability. The log-return formulation is numerically stable and scale-invariant across tickers.

Training episodes sample a random 252-day window from the training set, providing diverse market regimes per episode. Validation and test episodes always start at index 0 for reproducibility. The environment tracks position history, trades, and portfolio value, and computes Sharpe ratio, max drawdown, and trade frequency at episode end.

4 Deep Learning Methods

DQN approximates $Q^*(s, a)$ with a neural network Q_θ trained via Huber (smooth-ℓ_1) TD loss:

$$\mathcal{L} = \mathbb{E}_{\mathcal{D}}\Big[\text{SmoothL1}\big(r + \gamma \max_{a'} Q_{\bar{\theta}}(s', a'),\ Q_\theta(s, a)\big)\Big] \quad (2)$$

Key components (all from scratch): replay buffer (50K capacity, batch size 128), soft target network update (τ=0.005 per step), and exponential ε-decay from 1.0 to 0.02 with decay constant 20K steps. Rewards are clipped to $[-1, 1]$ before training to bound the TD target.

DDQN decouples action selection from evaluation to reduce overestimation:

$$y_t = r + \gamma\, Q_{\bar{\theta}}\big(s',\ \arg\max_{a'} Q_\theta(s', a')\big) \quad (3)$$

DDQN subclasses DQN, changing only the target computation.

Architecture. Both use a 3-layer MLP: $10 \to 128 \to 64 \to 3$ with ReLU activations, Adam (lr=5×10^{-4}), and gradient clipping (max norm 1.0).

5 Training and Results

Each agent trains for 200 episodes, each sampling a random 252-day window from the training set (2034 days) to expose the agent to diverse market regimes. Validation uses a deterministic greedy rollout (pure exploitation, from day 0) every 10 episodes; the checkpoint with the best validation score (cumulative return, then Sharpe) is restored for final test evaluation. DQN shows a clear reduction in training loss over time; DDQN convergence is noisier due to more conservative Q-value estimates.

Table 1: Test performance on AAPL (437 days, 2018–2020).

	DQN	**DDQN**	**B&H**
Cum. Return	+49.6%	−6.7%	+159.0%
Sharpe	0.84	0.03	1.65
Max DD	−32.4%	−38.7%	−31.4%

A detailed episode walkthrough showing position

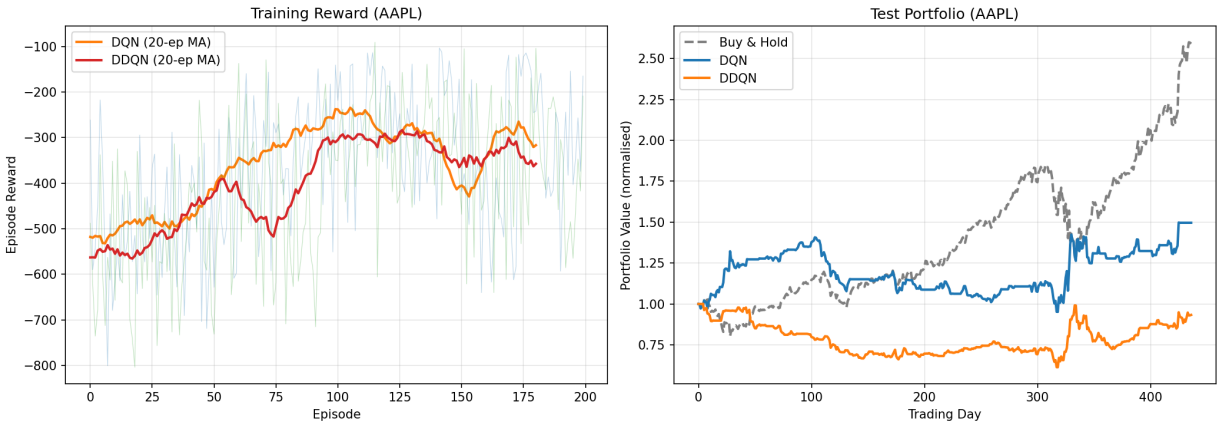

Figure 3: Left: Training reward (20-episode moving average). Right: Test portfolio value for DQN, DDQN, and buy-and-hold.

decisions, cumulative reward, and trade markers is provided in Appendix B (Figure 6).

The test period (2018–2020) includes the COVID-19 market crash of early 2020, making it a severe out-of-sample stress test. Buy-and-hold benefits strongly from AAPL's long-term appreciation ($\sim 3\times$ over the period), which is difficult to replicate through active trading. DQN achieves a positive cumulative return of +49.6% but falls short of the buy-and-hold baseline, suggesting the agent partially learns to avoid the worst drawdowns but cannot fully capture the sustained uptrend. DDQN underperforms, ending with a slight loss; its conservative Q-value estimates lead to excessive caution and missed long positions during the recovery. Both agents exhibit higher max drawdown than buy-and-hold, indicating the reward shaping was insufficient to fully protect against the sudden crash. These results highlight a fundamental challenge in RL-based trading: a strong passive baseline driven by secular growth is hard to beat with short-horizon signals alone.

6 Challenges

An early bug used normalised returns (std$\approx$1) instead of raw returns for rewards, producing unrealistic values; fixed by separating `raw_return`. Initial linear ε-decay exhausted exploration too quickly; switched to exponential decay for smoother exploration–exploitation trade-off. MSE loss was sensitive to large reward outliers; replaced with Huber (smooth-ℓ_1) loss and added reward clipping to $[-1, 1]$. Hard target-network copies caused training instability; replaced with soft polyak updates (τ=0.005). Short-selling can cause portfolio values below zero, so we clamp at 10^{-8}.

7 Future Work

Extensions include policy gradient methods (PPO) for continuous position sizing, multi-asset portfolio environments, LSTM architectures for temporal dependencies, and Sharpe/CVaR-based reward formulations.

8 Contributions

Member	Contributions
Rutwik	DQN/DDQN implementation from scratch; hyperparameter tuning (lr, batch size, buffer size); exponential ε-decay; soft target-network updates (τ=0.005); Huber loss and reward clipping; position-delta action space; log-return reward; random episode windows; best-model checkpointing; baseline comparison integration
Yufan	Data pipeline: `yfinance` OHLCV download, 9-feature engineering (RSI, MACD, Bollinger Bands, ATR, OBV, configurable MA deviations, volume normalisation), train-only z-score normalisation, multi-ticker `load_all()` support. `TradingEnv`: drawdown-penalised reward shaping (Eq. 1), flat & proportional transaction cost models, built-in Sharpe/drawdown/trade-frequency evaluation, position and portfolio tracking, episode logic
Fei Xin	Evaluation framework, training visualisation, report, and slides

References

[1] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, 518:529–533, 2015.

[2] H. van Hasselt et al., "Deep reinforcement learning with double Q-learning," *AAAI*, 2016.

[3] H. Yang et al., "Deep reinforcement learning for automated stock trading: An ensemble strategy," *ACM ICAIF*, 2020.

A Data Pipeline Visualisations

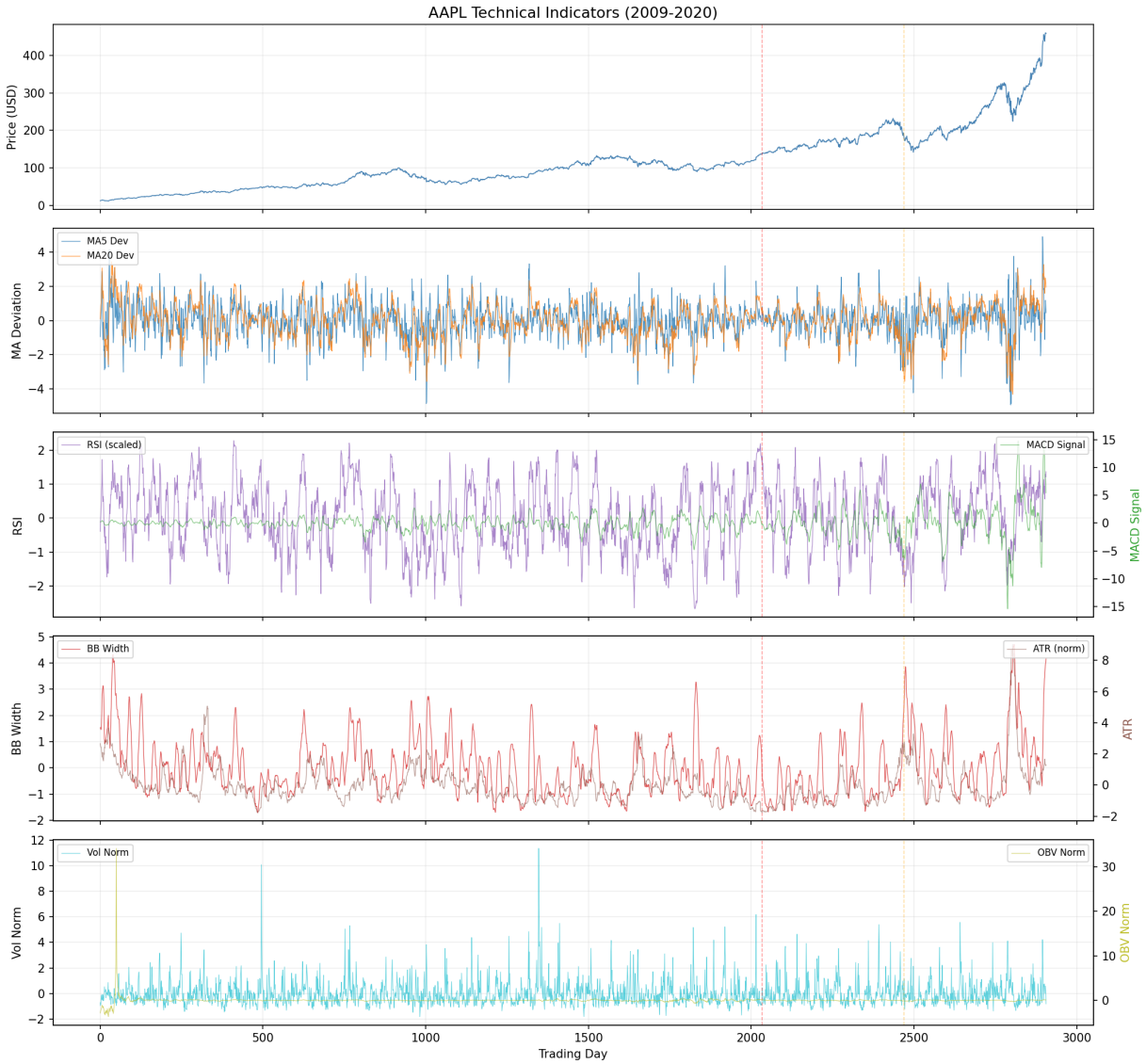

Figure 4: Technical indicator time series for AAPL (2009–2020). From top: close price, MA deviations, RSI & MACD, Bollinger width & ATR, volume & OBV. Dashed lines mark train/val/test boundaries.

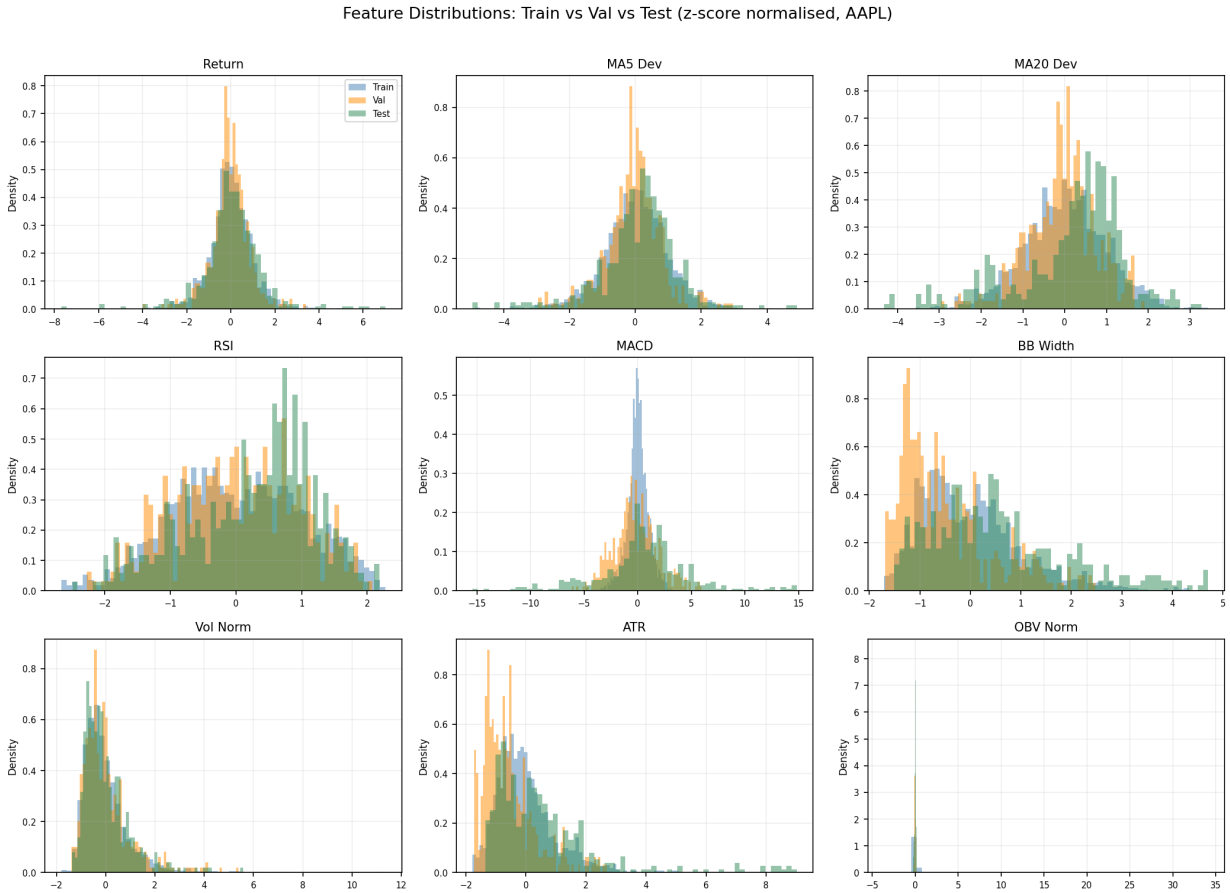

Figure 5: Feature distributions across train/val/test splits (z-score normalised using training-set statistics only). Overlapping distributions confirm consistent normalisation without look-ahead bias.

B Episode Walkthrough

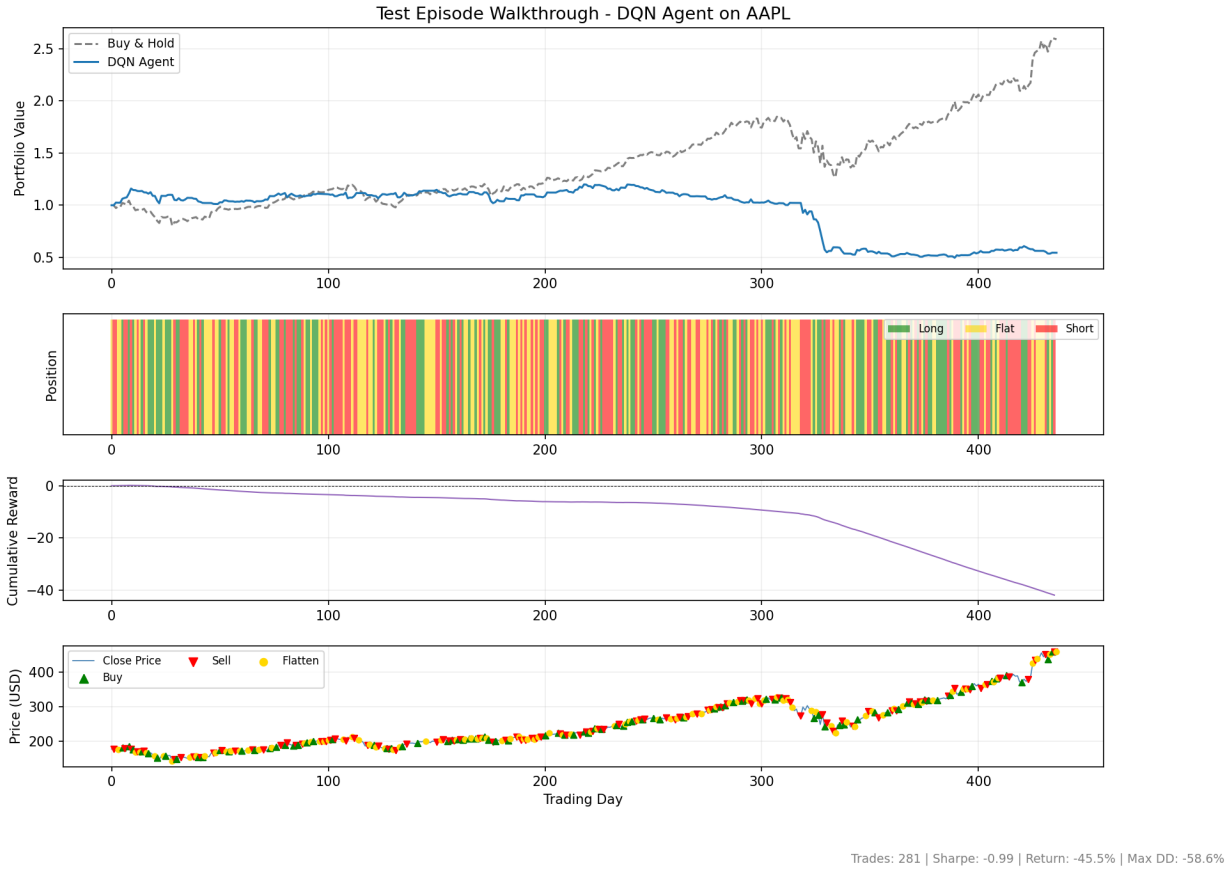

Figure 6: DQN test episode walkthrough on AAPL. Top: portfolio value vs buy-and-hold. Second: position colour bands (green=long, gold=flat, red=short). Third: cumulative reward. Bottom: buy/sell markers on close price.